

Rapport d’audit cybersécurité ICT — Backend Arquantix API

Référence technique : `services/arquantix/api`
Type de document : audit technique fondé sur le code source et les migrations versionnées ; ne constitue pas une certification DORA, une déclaration réglementaire ni une opinion juridique.
Règle de preuve : toute affirmation non étayée par un fichier ou une table du dépôt est marquée **NOT FOUND IN CODEBASE**.

1. Executive Summary

1.1 Finalité du système

L'API Arquantix fournit des services backend (authentification, sessions, opérations sensibles incluant custody et transferts internes) pour des opérateurs et flux métier reliés à des identités `AdminUser` liées à des personnes physiques. La surface d’authentification repose sur des jetons JWT (accès et refresh), des sessions persistantes en base, et des mécanismes optionnels de confiance device, d’attestation, de signature ECDSA et de scoring de risque multi-couches.

1.2 Posture de sécurité (synthèse factuelle)

Le code met en œuvre une **défense en profondeur partielle et conditionnelle** : les mécanismes les plus avancés (moteur de risque PR F, règles dynamiques, pseudo-ML, couche temporelle, moteur d’intention) sont **derrière des variables d’environnement** dont les valeurs par défaut désactivent souvent la fonction (voir section 11). La rotation des refresh tokens et la table des JTIs consommés constituent un socle solide contre la réutilisation naïve des jetons. En revanche, l’application uniforme du scoring PR F sur toutes les routes métier sensibles n’est **pas** réalisée : la dépendance `FastAPI require_low_risk_action` n’apparaît que dans `services/custody/router.py` (cinq endpoints), ce qui crée une **hétérogénéité de couverture** documentée en section 5.

1.3 Nomenclature des « PR » dans le dépôt

Les évolutions sont nommées dans les migrations et commentaires (PR D2 à PR F.7.2, etc.). Il n’existe pas de libellé unique « PR A » ou « PR G » dans un manifeste central. Pour ce rapport : **PR A** désigne la chaîne sessions + rotation refresh (migrations `108`, `131`, logique `refresh_session.py`) ; **PR G** désigne l’usage optionnel de Redis pour cache, rate limits et garde-fous distribués ; **PR H** n’est pas une migration séparée : le fichier `device_risk_engine_pr_f2.py` qualifie explicitement les **règles combinées non linéaires** de « PR H » au sein de `evaluate_combination_rules`, activées par `DEVICE_RISK_ENABLE_COMBINATION_RULES`. **NOT FOUND IN CODEBASE** : fichier ou révision Alembic intitulé uniquement « PR H ».

1.4 Évaluation de maturité globale (justification)

Une échelle qualitative **1 à 5** est appliquée par domaine en section 14. La **maturité globale pondérée** proposée est **3,8 / 5** : le code offre des capacités d’ingénierie avancées (sessions, signatures, attestation, empilement risque), mais l’effectivité en production, la couverture route par route, et la gouvernance hors code ne sont pas démontrables depuis le seul dépôt. Cette note est une **indication technique**, pas un score de conformité DORA.

1.5 Layered Defense Model (vue synthétique)

Le système implémente une **architecture de défense en profondeur** empilant des mécanismes partiellement redondants : la défaillance d’un seul contrôle ne doit pas, en conception, suffire à compromettre l’ensemble du périmètre — sous réserve que les flags soient activés et les routes couvertes (voir sections 5 et 6.8).

Ordre	Couche	Mécanisme principal (réf. code)
1	Identité	Résolution de sujet JWT, rattachement utilisateur (<code>auth.py</code> , <code>jwt_subject_resolution.py</code>)
2	Sécurité de session	Rotation refresh, suivi des JTI, révocation (<code>refresh_session.py</code> , <code>auth_spent_refresh_jti</code>)
3	Liaison device	X-Device-ID , profils <code>auth_user_device_profiles</code> , cohérence session
4	Intégrité de requête	ECDSA, nonces, portée par route (PR D2–D4)
5	Confiance device	Attestation matérielle / logicielle (<code>device_attestation_*</code>)
6	Analyse comportementale	Pile moteur de risque PR F (F.2 à F.7.2 selon activation)
7	Détection d’intention	PR F.6, motifs sur historique d’événements
8	Contrôles distribués	Redis optionnel : cache, rate limiting, anti-replay nonces

Lecture régulateur / investisseur : cette pile offre une **vision d'ensemble** avant le détail architectural (section 3). Elle ne préjuge pas de l'**effectivité opérationnelle** : celle-ci est traitée en section 6.8.

2. Scope & Methodology

2.1 Périmètre du codebase

Inclus : arborescence Python sous `services/arquantix/api`, modèles SQLAlchemy dans `database.py`, migrations sous `alembic/versions/`, tests sous `tests/`. Exclus : déploiement réel, configuration des secrets, réseau (TLS, pare-feu), clients mobiles, autres services du monorepo non importés par cette API.

2.2 Méthode d'analyse

Analyse statique : lecture des routeurs principaux (`main.py`, routeurs inclus), des modules `services/auth/*` et `services/security/*` pertinents, recherche globale des symboles (`require_low_risk_action`, `evaluate_pr_f_for_request`, tables `auth_*`). Croisement avec les révisions Alembic récentes 131–142 pour les contrôles device et risque.

2.3 Limitations

Aucune exécution dynamique sur un environnement de production ; aucune vérification que les flags sont positionnés conformément à une politique interne. Les revendications légales « DORA compliant » au sens juridique sont **hors scope** : le présent document mappe des **capacités techniques** à des domaines de contrôle ICT usuels, sans interpréter le texte légal article par article.

3. System Architecture

3.0 Renvoi au modèle défensif

La **vue synthétique en huit couches** est présentée en section 1.5. Les sous-sections 3.1 à 3.8 détaillent l'implémentation correspondante et les flux de confiance.

3.1 Couche identité et sujets JWT

Les sujets JWT sont résolus vers des utilisateurs applicatifs via les modules d'authentification (`auth.py`, `jwt_subject_resolution.py`). Les jetons d'accès portent des claims incluant notamment l'identifiant de session (`sid`) lorsque la session Phase 2 est utilisée, ce qui relie l'identité au modèle `AuthSession`.

3.2 Gestion de session et refresh

Les sessions sont stockées dans `auth_sessions` (migration 108, évolutions ultérieures). La rotation des refresh tokens s'appuie sur `auth_refresh_tokens` (migration 131) et sur `auth_spent_refresh_jti` pour enregistrer les JTIs consommés et détecter les réutilisations. La logique procédurale est centralisée dans `services/auth/refresh_session.py` (`perform_login`, `perform_refresh`, `perform_revoke`, etc.).

3.3 Liaison device

Le lien entre utilisateur et appareil repose sur `X-Device-ID` normalisé (`normalize_device_id`), sur les profils `auth_user_device_profiles` (migration 118), et sur les champs `device_id` / empreintes dans `AuthSession`. Le moteur PR F exige un device non « legacy-unknown » lorsque `require_low_risk_action` est actif (`device_risk_pr_f_dependencies.py`).

3.4 Signature ECDSA (PR D2–D4)

Les clés publiques sont stockées dans `auth_device_credentials`. Les nonces de signature sensible sont dans `auth_device_signature_nonces` avec extension de portée par route (134). La vérification des requêtes sensibles passe par `device_sensitive_signature.py` et politiques dans `device_pr_d3_policy.py` / `device_security_pr_d2.py` (niveau `DEVICE_SECURITY_LEVEL`).

3.5 Attestation

Les en-têtes d'attestation sont traités par `device_attestation_service.py` et les dépendances `device_attestation_dependencies.py`. Des champs d'état d'attestation sont portés par `AuthSession` ; des migrations dédiées incluent 112 et 135.

3.6 Pile moteur de risque

L'évaluation principale est `evaluate_pr_f_for_request` dans `device_risk_engine_pr_f.py`. L'ordre logique après résolution du cache éventuel est : **règles dynamiques (F.4)** → **règles combinées « PR H » (F.2)** → **bonus baseline géographique et score temporel F.3** →

score de base pondéré ou legacy → overlay ML (F.7) → overlay temporel (F.7.2) → décision `decide_risk_action` . Ensuite, le résultat transite par `_risk_finish` qui invoque `evaluate_intent_engine` (F.6) si activé, pouvant surclasser la décision.

3.7 Redis et infrastructure distribuée

Redis est optionnel (`REDIS_ENABLED` et dérivés dans `security_env.py`) : cache identité, rate limiting distribué, replay nonces, override dry-run des règles. **NOT FOUND IN CODEBASE** : configuration Terraform/Helm pour Redis dans ce dépôt.

3.8 Flux de données et frontières de confiance

Frontière nord : client HTTP (en-têtes `Authorization` , `X-Device-ID` , `X-Forwarded-For` , `CF-IPCountry` , attestation, signatures). Le serveur **fait confiance** à ces en-têtes pour le scoring et la géolocalisation dérivée : c'est une hypothèse d'infrastructure (terminaison TLS, proxy de confiance). **Frontière sud** : PostgreSQL comme source de vérité pour sessions et règles ; Redis si activé comme cache / verrou distribué.

4. Security Control Framework

Le tableau suivant résume, pour chaque couche, l'objectif de contrôle, l'implémentation vérifiable, la menace adressée et le risque résiduel.

Couche	Objectif de contrôle	Implémentation (références)	Menace atténuée	Risque résiduel
Identité	Assurer que les actions authentifiées se rapportent à un utilisateur connu	<code>auth.py</code> , <code>jwt_subject_resolution.py</code> , <code>AdminUser</code>	Usurpation d'identité par sujet JWT mal formé	Vol de secret de signature JWT ; compromission du <code>SECRET_KEY</code> (hors code)
Session	Limiter l'abus des jetons de longue durée	<code>refresh_session.py</code> , <code>AuthSession</code> , <code>AuthSpentRefreshJti</code>	Rejeu de refresh	Courses parallèles documentées dans le code ; horloge
Device	Lier l'activité à un appareil connu	<code>AuthUserDeviceProfile</code> , PR F exige <code>X-Device-ID</code>	Device spoofing partiel	Clients <code>Legacy-unknown</code> ; incohérence de couverture PR F
Signature	Intégrité et fraîcheur des requêtes sensibles	<code>device_sensitive_signature.py</code> , PR D3/D4	Replay API, MITM sur payload	Niveau 0 de <code>DEVICE_SECURITY_LEVEL</code> : pas d'exigence de signature par cette politique
Attestation	Ancrer la confiance matérielle / OEM	<code>device_attestation_service.py</code> , champs session	Clones logiciels, émulateurs	Qualité variable selon client ; pas de garantie physique absolue
Moteur de risque	Agréger signaux et décider allow / step_up / block	<code>device_risk_engine_pr_f.py</code> et modules liés	Fraude comportementale, scénarios combinés	Heuristiques ; flags désactivés ; routes sans <code>require_low_risk_action</code>
Redis / infra	État partagé pour limites et cache	<code>redis_client.py</code> , <code>security_env.py</code>	Déni de service distribué	Indisponibilité Redis ; mauvaise segmentation réseau (hors code)

5. Control Coverage Analysis

5.1 Méthode

Recherche exhaustive de `require_low_risk_action` dans `services/arquantix/api/**/*.py` . Résultat : **six** occurrences — la définition dans `device_risk_pr_f_dependencies.py` et **cinq** injections dans `services/custody/router.py` .

5.2 Matrice de couverture (routes où PR F unifié est appliqué)

Route (préfixe router)	Handler (résumé)	<code>require_low_risk_action</code>
<code>POST /api/admin/custody/accounts/client</code>	Création compte client	Oui
<code>POST /api/admin/custody/accounts/client/canonical</code>	Création canonique	Oui
<code>POST /api/admin/custody/webhook-events/{id}/replay</code>	Replay webhook	Oui
<code>POST /api/admin/custody/simulate-withdrawal</code>	Simulation retrait	Oui
<code>POST /api/internal-transfer</code>	Transfert interne	Oui

5.3 Écart identifié (même famille métier)

Le endpoint `POST /api/admin/custody/accounts/client/simple-create` applique `require_continuous_auth_for_action("beneficiary_add")` et `require_device_attestation()` mais **n'inclut pas**

`Depends(require_low_risk_action())` d'après le fichier `custody/router.py` analysé. Les autres créations de compte client sur le même routeur incluent PR F. **C'est un écart de couverture explicite.**

5.4 Autres routes sensibles sans PR F unifié

Les endpoints d'authentification (`/auth/login`, `/auth/refresh`, `/auth/revoke`), les routes passkeys sous `/auth/passkeys`, et les flux OTP (`admin_email_otp_routes.py`, etc.) **ne** montent **pas** `require_low_risk_action` dans le périmètre analysé. Ils s'appuient sur d'autres mécanismes (rotation, middlewares globaux, auth continue). **NOT FOUND IN CODEBASE** : autre fichier que `custody/router.py` important `require_low_risk_action`.

5.5 Plan de remédiation (recommandation technique)

Priorité	Action
Haute	Décider si <code>simple-create</code> doit aligner ses <code>Depends</code> sur les autres routes <code>beneficiary_add</code> ; si oui, ajouter <code>require_low_risk_action()</code> .
Haute	Maintenir une matrice signée produit / sécurité listant chaque route sensible et ses dépendances.
Moyenne	Documenter explicitement pourquoi login/refresh ne doivent pas invoquer PR F (ou ajouter un pré-filtre léger si la menace l'exige).

6. Risk Engine Deep Analysis

6.1 PR F — score de base

Logique : `compute_risk_score` agrège confiance device, état d'attestation, cohérence réseau, vitesse, échecs de signature, etc. **Forces** : explicabilité via raisons textuelles ; plafonnement du score. **Faiblesses** : sensibilité aux signaux fournis par en-têtes ; score sans PR F sur de nombreuses routes. **Surface d'attaque** : manipulation d'en-têtes si bordure non fiable.

6.2 PR F.2 et PR H (règles combinées)

Logique : `evaluate_combination_rules` dans `device_risk_engine_pr_f2.py` ; commentaire de code « Règles non linéaires (PR H) ». Activation : `DEVICE_RISK_ENABLE_COMBINATION_RULES`. **Forces** : court-circuits stricts (ex. nouveau device et changement de pays). **Faiblesses** : désactivé par défaut ; règles statiques codées. **Surface** : contournement si combinaisons ne couvrent pas un scénario réel.

6.3 PR F.3 — baseline avancée

Logique : `baseline_temporal_anomaly_score` et mises à jour dans `device_risk_engine_pr_f3.py`, persistance `auth_user_risk_baselines`. **Forces** : agrégats temporels (Welford). **Faiblesses** : besoin d'échantillons ; faux positifs sur changement de mode de vie.

6.4 PR F.4 — règles dynamiques

Logique : `evaluate_dynamic_rules` ; table `auth_risk_rules` ; DSL JSON validé par `validate_risk_rule_conditions`. **Forces** : évolutivité sans redéploiement. **Faiblesses** : erreur humaine sur règles valides syntaxiquement mais dangereuses ; dry-run peut masquer une erreur de configuration si mal gouverné.

6.5 PR F.6 — intent engine

Logique : `evaluate_intent_engine` ; motifs sur historique récent ; peut forcer `block` ou `step_up` ; journal `auth_user_intent_events` via `log_intent_event` dans la chaîne PR F. **Forces** : séquences métier explicites. **Faiblesses** : fenêtre temporelle fixe en code ; contournement par lenteur extrême.

6.6 PR F.7 — pseudo-ML

Logique : `device_risk_ml_engine.py` ; features dérivées d'événements intent et session intelligence ; z-score vs EMA ; pas de bibliothèque ML externe dans ce module. **Forces** : pas de boîte noire externe ; gel EMA si score pré-ML élevé (F.7.1). **Faiblesses** : calibration empirique ; défaut `DEVICE_RISK_ML_ENABLED=false`.

6.7 PR F.7.2 — temporel

Logique : `device_risk_temporal_features.py` / `device_risk_temporal_engine.py` ; distributions heure/jour/transitions ; table `auth_user_temporal_features`. **Forces** : explain strings ; plafond de contribution. **Faiblesses** : seuils internes fixes ; min samples configurable.

6.8 Control Effectiveness (Technical Assessment)

L'**effectivité** d'un contrôle ne se confond pas avec son **existence** dans le code. Elle dépend de facteurs que seule l'exploitation peut valider pleinement ; le tableau ci-dessous donne une **appréciation technique** fondée sur la logique implémentée et les dépendances déjà

identifiées (sections 5 et 11).

Facteurs d'effectivité communs : (1) activation des feature flags et valeurs de seuils ; (2) montage correct des dépendances FastAPI sur chaque route sensible ; (3) qualité des signaux amont (X-Device-ID , en-têtes IP/pays derrière un proxy de confiance).

Contrôle ou famille	Effectivité technique observée (à froid, code)	Facteurs limitants
Rotation des refresh tokens et détection de reuse	Élevée — logique persistée (<code>auth_spent_refresh_jti</code> , révocation)	Fenêtres concurrentielles documentées ; horloge serveur
Liaison device (<code>device_id</code> , profils)	Moyenne — forte lorsque PR F est monté et X-Device-ID présent	Intégrité du <code>device_id</code> côté client ; valeur <code>legacy-unknown</code>
Moteur de risque PR F (pile complète)	Variable — puissant lorsque activé sur la route	Défauts de flags à <code>false</code> ; écart de couverture (ex. <code>simple-create</code>)
Signature ECDSA + nonces sur routes sensibles	Conditionnelle — dépend de <code>DEVICE_SECURITY_LEVEL</code> et politiques PR D3	À niveau 0, pas d'exigence de signature sensible via la politique documentée dans <code>device_pr_d3_policy.py</code>
Attestation	Moyenne à élevée selon client	Variabilité des preuves OEM ; pas de garantie universelle
Règles dynamiques F.4	Variable — dépend de <code>DEVICE_RISK_ENABLE_DYNAMIC_RULES</code>	Erreurs de conception de règle malgré DSL valide
Couches F.7 / F.7.2 / F.6	Variable — dépend des flags respectifs	Volume de données et calibration
Redis (rate limit, cache, nonces)	Conditionnelle — utile si <code>REDIS_ENABLED</code> et réseau sain	Indisponibilité ou exposition réseau du datastore

Conclusion (lecture opérationnelle) : les mécanismes sont **robustes sur le plan de l'ingénierie logicielle**, mais l'**effectivité en production** repose sur une configuration cohérente, une **couverture de routes complète** là où le risque l'exige, et une **infrastructure** qui préserve la confiance dans les en-têtes et l'isolation Redis. Sans preuve runtime (section 13), cette appréciation reste **technique**, non **mesurée** sur trafic réel.

7. Threat Model & Attack Simulation

Les scénarios ci-dessous décrivent le **comportement attendu d'après le code**, pas des résultats de pentest annexés (**NOT FOUND IN CODEBASE** pour un rapport d'intrusion).

7.1 Vol de refresh token après rotation

Étapes : attaquant réutilise un JTI déjà marqué consommé. **Réaction** : insertion ou détection sur `auth_spent_refresh_jti` , révocation de session, erreur HTTP 401 avec message de reuse (`refresh_session.py`). **Résultat attendu** : session compromise clôturée. **Faiblesse résiduelle** : conditions de concurrence documentées dans les commentaires du module refresh.

7.2 Spoofing device / IP / pays

Étapes : requêtes avec en-têtes falsifiés depuis un client contrôlé. **Réaction** : scoring PR F et baselines utilisent ces signaux. **Résultat** : possible adaptation du score si incohérences. **Faiblesse** : si le proxy n'est pas digne de confiance, les signaux sont toxiques.

7.3 Fraude lente

Étapes : attaquant imite le rythme historique. **Réaction** : F.7 / F.7.2 peuvent ne pas dévier fortement. **Faiblesse** : faux négatifs inhérents aux heuristiques.

7.4 Contournement de règle

Étapes : accès à une route sans `require_low_risk_action` alors qu'une route voisine l'a. **Réaction** : aucune évaluation PR F sur cette route. **Faiblesse** : **gap architectural** (ex. `simple-create`).

7.5 Manipulation Redis

Étapes : effacement de clés de cache ou de nonce si Redis exposé. **Réaction** : dépend du déploiement ; le code ne sécurise pas le réseau Redis. **Faiblesse** : **NOT ASSESSABLE IN CODEBASE** — segmentation réseau et ACL Redis.

8. DORA Mapping (capacités techniques uniquement)

Aucune allégation de conformité légale. Correspondance indicative :

Domaine de contrôle ICT (formulation générique)	Capacités présentes dans le code
Gestion des risques ICT	Flags, seuils, règles paramétrables, simulation
Contrôle d'accès	JWT, sessions, rôles, auth continue, Zero Trust sur certaines routes
Détection d'incidents / anomalies	Moteur de risque, logs structurés, tables d'audit
Résilience opérationnelle	Dépendances optionnelles Redis ; persistance PostgreSQL
Traçabilité	Tables <code>auth_*</code> , événements sécurité si activés

9. Evidence & Logging

9.1 Tables d'audit pertinentes (liste non exhaustive)

`auth_sessions`, `auth_refresh_tokens`, `auth_spent_refresh_jti`, `auth_user_device_profiles`, `auth_device_credentials`, `auth_device_signature_nonces`, `auth_user_risk_baselines`, `auth_risk_rules`, `auth_user_intent_events`, `auth_user_risk_features`, `auth_user_temporal_features`, `auth_security_events` (si persistance activée).

9.2 Journaux représentatifs

Exemples de noms d'événements ou de logs : `device_risk_evaluated`, `refresh_token_reuse_detected`, `device_risk_temporal_evaluated`, `device_intent_engine_block`, `device_intent_engine_step_up`. La présence effective dans les agrégateurs de logs de production n'est pas vérifiable dans le dépôt.

10. Residual Risks

Limites architecturales : hétérogénéité de la couverture PR F ; intent appliqué après le calcul PR F dans `_risk_finish`, ce qui peut surclasser une décision mais ne s'applique pas aux routes sans chaîne PR F.

Limites anti-fraude comportementale : heuristiques non calibrées par un tiers ; dépendance à la volumétrie d'événements intent.

Dépendances de configuration : l'essentiel des défenses avancées est désactivé tant que les variables listées en section 11 restent à leurs valeurs par défaut « off ».

11. Runtime Configuration Requirements

Les **valeurs par défaut** ci-dessous proviennent de `security_env.py` et des modules PR D2 sauf mention contraire. Les **valeurs requises** pour une posture « défense activée » sont des **recommandations d'exploitation** ; elles ne sont pas prescrites par le seul dépôt.

Variable	Défaut (code)	Valeur typique recommandée (exploitation)	Impact si désactivé ou à 0
<code>DEVICE_SECURITY_LEVEL</code>	<code>0</code> (<code>device_security_pr_d2.py</code>)	2 ou 3 si signatures et politiques étendues requises	Pas d'exigence de signature sensible via <code>sensitive_routes_device_signature_enabled</code> (faux si niveau inférieur à 2)
<code>DEVICE_RISK_ENGINE_PR_F_ENABLED</code>	<code>false</code>	<code>true</code> sur routes protégées	Aucune évaluation PR F sur routes qui appellent le Depends
<code>DEVICE_RISK_ENABLE_DYNAMIC_RULES</code>	<code>false</code>	<code>true</code> si règles DB	Règles <code>auth_risk_rules</code> non appliquées dynamiquement
<code>DEVICE_RISK_ML_ENABLED</code>	<code>false</code>	<code>true</code> si couche F.7 voulue	Pas d'overlay ML
<code>DEVICE_RISK_TEMPORAL_ENABLED</code>	<code>false</code>	<code>true</code> si F.7.2 voulue	Pas de scoring temporel
<code>REDIS_ENABLED</code>	<code>false</code>	<code>true</code> si besoin distribué	Cache et rate limits distribués limités

Variables additionnelles documentées dans le code : `DEVICE_INTENT_ENGINE_ENABLED`, `DEVICE_RISK_ENABLE_COMBINATION_RULES`, `DEVICE_RISK_RULES_DRY_RUN`, `AUTH_SECURITY_EVENTS_ENABLED` (via modules concernés), etc.

12. Governance & Operations

Gestion des règles : API `/admin/risk` dans `risk_rules_admin_routes.py` : liste, création, patch, validation DSL (POST `/rules/validate`), simulation. Champs `enabled`, `is_active`, `ruleset`, `priority`, `version`.

Contrôles de gouvernance absents du code : workflow d'approbation à quatre yeux obligatoire ; séparation stricte des rôles sur les mutations de règles — **NOT FOUND IN CODEBASE** comme exigence codée ; seul `get_current_user` borne l'accès admin.

13. Required Evidence Package (hors dépôt)

Pour répondre à une demande régulateur ou due diligence, les éléments suivants doivent être **assemblés en annexe** ; ils ne sont pas générés par Git seul.

Artéfact	Contenu attendu
Snapshot de configuration	Variables d'environnement non secrètes pour les flags section 11
Journaux d'exploitation	Échantillons datés des événements listés section 9
Rapport red team / pentest	NOT FOUND IN CODEBASE — à produire séparément
Schéma d'infrastructure	TLS, placement Redis, accès DB, WAF
Procédure de changement	Tickets, CAB, approbations pour règles et seuils

14. Final Maturity Assessment

Notation **0 à 5** par domaine (5 = capacité maximale observée au niveau **code**, indépendamment de la preuve production).

Domaine	Score	Justification succincte
Identité & JWT	4,0	Modèles et résolution structurés ; dépendance aux secrets hors repo
Sessions & refresh	4,5	Rotation, spent JTI, tests auth
Device & profils	4,0	Profils et PR F ; chemin legacy résiduel
Signature & nonces	3,5	Complet en code ; activation par <code>DEVICE_SECURITY_LEVEL</code>
Attestation	3,5	Intégration présente ; variabilité client
Moteur de risque (pile F)	4,0	Riche mais flags et couverture route
Redis / distribué	3,0	Optionnel ; pas de preuve infra
Gouvernance des règles	2,5	API admin ; pas de workflow d'approbation codé
Observabilité	3,5	Hooks nombreux ; désactivation par flags

Score final (moyenne simple des neuf domaines) : 3,7 / 5.

Clause de clôture : ce score mesure la **maturité d'implémentation logicielle** telle que visible dans le dépôt. Il ne constitue pas une notation de conformité DORA, ni un substitut à un audit tiers.

Fin du rapport.